IPC: Shared Memory

Inter-Process Communication (System V) Simulation Trace

Shared Memory is the fastest form of Inter-Process Communication (IPC). It allows two or more isolated processes to map the same physical memory segment into their respective virtual address spaces, enabling instant data exchange without kernel overhead.

The System V IPC API

Before running the code, understanding the four core system calls used in C programming is essential:

```
shmget()
-> Allocates a System V shared memory segment and returns its ID.

shmat()
-> Attaches the shared memory segment to the process's address space.

shmdt()
-> Detaches the segment from the process's address space.

shmctl()
-> Performs control operations, such as deleting the segment (IPC_RMID).
```

1. Executing the Writer Process

We execute a C program (`writer.c`) that generates a unique key, allocates 1024 bytes of shared memory, maps it, and writes a payload.

```
user@os-sim:~/ipc$ gcc writer.c -o writer
user@os-sim:~/ipc$ ./writer
[Writer] Generating unique IPC key via ftok()... Key: 0x41048b03
[Writer] Requesting 1024 bytes of shared memory via shmget()...
[Writer] Shared Memory Segment Created. ID: 32768
[Writer] Attaching segment to virtual memory (shmat)... Mapped at address
0x7f9b8c000000
[Writer] Writing payload to memory...
[Writer] Data "CRITICAL_PAYLOAD_A: OS_TICK_9942" successfully written.
[Writer] Detaching memory (shmdt). Process exiting.
```

2. Inspecting OS Resources with `ipcs`

Even though the writer process has exited, the shared memory segment persists in the kernel. We use the `ipcs -m` command to view all active shared memory segments in the system.

```
user@os-sim:~/ipc$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch
status
0x00000000 131072     root       600        4194304    2
dest
0x41048b03 32768      user       666        1024       0

# Notice our segment (32768) is alive. 'nattch' is 0 because the writer
detached.
```

3. Executing the Reader Process

Now, an entirely separate process (`reader.c`) is launched. It uses the same key to locate the memory segment, attaches it, reads the data, and then asks the kernel to destroy the segment.

```
user@os-sim:~/ipc$ gcc reader.c -o reader
user@os-sim:~/ipc$ ./reader
[Reader] Locating Shared Memory Segment using Key: 0x41048b03...
[Reader] Segment ID 32768 found.
[Reader] Attaching segment to virtual memory (shmat)... Mapped at address
0x7f2a11000000
[Reader] Reading data from shared memory pointer:
         >> "CRITICAL_PAYLOAD_A: OS_TICK_9942"
[Reader] Detaching memory (shmdt).
[Reader] Issuing shmctl(IPC_RMID) to destroy the segment. Process exiting.
```

4. Verifying Cleanup

Shared memory must be manually deleted (via code or the `ipcrm` command) to prevent memory leaks. We check `ipcs -m` again to ensure the reader successfully destroyed the segment.

```
user@os-sim:~/ipc$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch
status
0x00000000 131072     root       600        4194304    2          dest

# Our segment 32768 is gone. The system memory has been freed.
```

5. Under the Hood with `strace`

We can use `strace` to monitor the actual system calls made by the reader process as it interacts with the kernel to access the memory.

```
user@os-sim:~/ipc$ strace -e shmget,shmat,shmdt,shmctl ./reader
shmget(0x41048b03, 1024, 0666)          = 32768
shmat(32768, NULL, 0)                   = 0x7f2a11000000
... (printf outputs omitted) ...

shmdt(0x7f2a11000000)                   = 0
shmctl(32768, IPC_RMID, NULL)           = 0
+++ exited with 0 +++
```